Reporte General Final del Proyecto TD3 para una Protesis Mioelectrica

Desde la base conceptual hasta la linea residual final sobre `Agent7250`

Proyecto `EMG_Prosthesis_TD3`

26 de marzo de 2026

Resumen

Este documento consolida la linea completa del proyecto de control mioelectrico con TD3, desde su base conceptual inicial hasta el estado final actual. El hilo conductor del reporte es simple: primero hubo que depurar reward, entorno y simulador para obtener un benchmark valido; despues hubo que intentar mejorar ese benchmark sin romper el tracking. La fase previa a la validacion dejo aprendizaje tecnico importante, pero el primer benchmark plenamente defendible aparecio despues del fix del simulador, cuando la corrida de 8000 episodios produjo a `Agent7250`. A partir de ese punto se abrio una nueva fase, ya no orientada a "hacer que aprenda", sino a reducir agresividad y esfuerzo manteniendo el tracking. Se probaron varias lineas de bajo riesgo: continuacion con menor exploracion, exploracion *threshold-aware*, regularizacion de saturacion, *warm-start* offline y una interfaz de accion alineada al actuador. Ninguna reemplazo al benchmark. La intervencion que si aporto una mejora real fue la politica residual sobre `Agent7250`. El mejor candidato final de esa familia fue `Agent1850` con $\alpha_{\text{res}} = 0.20$, cuyo test visual de 50 simulaciones obtuvo `trackingMSE = 0.043445`, `trackingMAE = 0.163542`, `actionL2 = 0.523597`, `saturationFraction = 0.272637` y `deltaActionL2 = 0.299262`. Ese resultado cumple `ConditionB` frente a `Agent7250` y se toma aqui como punto final operativo del proyecto.

Índice

1. Que documenta este reporte **3**

2. Base conceptual del proyecto **3**

3. Antes del benchmark valido: que hubo que corregir **3**
- 3.1. Reward heredada . . . 3
- 3.2. Paso a una reward basada en tracking MSE . . . 4
- 3.3. Accion efectiva cuantizada . . . 4
- 3.4. El bug del simulador . . . 4

4. Construccion del benchmark valido **4**
- 4.1. Baseline limpio post-fix . . . 4
- 4.2. De 2000 a 8000 episodios . . . 5

5. Por que fue necesario abrir una fase nueva despues de `Agent7250` **6**

6. Experimentos posteriores a `Agent7250` **6**
- 6.1. Continuacion con menor exploracion . . . 6
- 6.2. Exploracion threshold-aware . . . 7
- 6.3. Regularizacion por saturacion . . . 7
- 6.4. Warm-start offline . . . 7

6.5. Interfaz de accion alineada al actuador 7
6.6. Resumen comparativo 7

7. La fase nueva que si funciono: Residual TD3 **8**
7.1. Por que la idea residual tenia sentido 8
7.2. Que se implemento 8
7.3. Piloto residual principal 9
7.4. Por que no se eligio “mas episodios” 10
7.5. Que es alpha_res y por que hubo barridos 10
7.6. Resultado de los barridos 11
7.7. Que dicen los diagnosticos residuales 12

8. Agente final recomendado **13**

9. Conclusion final **13**

1. Que documenta este reporte

Este reporte tiene tres objetivos:

1. reconstruir la linea del proyecto desde la base conceptual hasta el estado final actual;
2. explicar de forma sencilla por que se tomo cada iniciativa importante;
3. dejar fijado, con criterio tecnico y con evidencia, cual es el mejor agente final de la linea actual.

La idea no es reescribir toda la historia como si todas las corridas hubieran valido igual. Al contrario: una parte importante del trabajo consistio precisamente en aprender a separar:

- exploracion util,
- resultados contaminados por problemas del entorno,
- y benchmark realmente valido.

2. Base conceptual del proyecto

La raiz conceptual del proyecto es el manuscrito *Continuous Operation of a Myoelectric Hand Prosthesis using the TD3 Reinforcement Learning Algorithm* [1]. Ese trabajo fijaba tres ideas que siguieron siendo correctas durante toda la evolucion posterior:

- el problema es de control continuo sobre cuatro motores;
- la politica debe mapear observaciones EMG y estado mecanico a acciones continuas;
- TD3 es una eleccion razonable para ese tipo de tarea [2, 3, 4].

Esa base tambien es coherente con la literatura general de control mioelectrico: la protesis no debe responder solo como un clasificador de gestos, sino como un sistema continuo, proporcional y razonablemente suave [5, 6].

3. Antes del benchmark valido: que hubo que corregir

3.1. Reward heredada

La primera version del repositorio usaba una reward heredada que mezclaba varias ideas a la vez: direccion correcta, castigo por error, incentivos a no quedarse quieto y penalizacion por cambios bruscos. El problema no era que esas ideas fueran absurdas; el problema era que estaban acopladas de forma poco interpretable. En la practica:

- trataba acciones continuas como si fueran casi discretas;
- su escala no hacia obvio que termino dominaba realmente;
- dejaba abierta la puerta a *reward hacking*;
- hacia dificil saber si el agente mejoraba tracking o solo aprendia a explotar huecos de la reward.

Ese tipo de problema es bien conocido en *reward shaping*: una reward compleja no necesariamente induce el comportamiento correcto si no esta alineada con el objetivo fisico real [7].

3.2. Paso a una reward basada en tracking MSE

Por eso se fue migrando a una reward mucho mas clara:

$$r_t = -\frac{1}{4}\sum_{i=1}^{4}\left(e_{t,i}^2 + \lambda_a \tilde{a}_{t,i}^2 + \lambda_{\Delta a}\left(\tilde{a}_{t,i} - \tilde{a}_{t-1,i}\right)^2\right),$$

donde el termino dominante es el error de tracking, y los otros dos solo regularizan esfuerzo y brusquedad.

La motivacion fue simple:

- si el proyecto busca seguimiento continuo, el error de seguimiento debe ser el centro de la reward;
- si se quiere evitar politicas violentas, eso debe entrar como regularizacion, no como objetivo principal;
- una formulacion asi es mas compatible con TD3 y mas interpretable durante la auditoria [2, 8].

3.3. Accion efectiva cuantizada

Otra mejora estructural fue explicitar que la accion realmente aplicada por el entorno no era una variable continua arbitraria, sino una accion efectiva cuantizada en niveles PWM utiles. Esta alineacion era necesaria porque el actor producia valores en $[-1, 1]$, pero el simulador y el actuador operaban con bins efectivos.

3.4. El bug del simulador

El hallazgo mas importante de toda la historia temprana fue que el simulador estaba roto. La dinamica dependia de `fit_C2.mat`, pero el contenido util para mover la protesis estaba vacio, de modo que muchas corridas previas no estaban controlando una planta que realmente se moviera. Ademas, habia un desfase temporal en `step`.

Consecuencia metodologica:

- las corridas previas al fix ya no pudieron usarse como benchmark numerico serio;
- si conservaron valor como etapa de depuracion de reward, logging y estructura del entorno.

Este punto es fundamental para entender todo el resto del proyecto: la comparacion valida solo empieza despues del fix del simulador.

4. Construccion del benchmark valido

4.1. Baseline limpio post-fix

Despues de corregir el simulador, el baseline se estabilizo con:

- observacion `markov52`,
- reward `trackingMseActionRateReward`,
- TD3 feedforward,
- remapeo efectivo cuantizado,
- entrenamiento y test en simulacion con dataset pregrabado.

La observacion `markov52` fue una extension razonable del estado base del articulo [1]: conserva las features EMG, pero anade informacion mecanica util como delta de encoder y accion previa.

4.2. De 2000 a 8000 episodios

La primera corrida post-fix plenamente util fue la de 2000 episodios. Luego se hizo una corrida larga de 8000 episodios y una extension a 12000. El resumen de esos hitos es:

Cuadro 1: Hitos validos de la linea post-fix antes de la fase residual

Agente / etapa	tracking MSE	tracking MAE	action L2	saturation Fraction	deltaAction L2
2000 episodios (nuevo baseline post-fix)	0.052805	0.179093	0.674957	0.474999	0.633264
`Agent7250` en 8000 episodios	0.043045	0.160336	0.596444	0.392086	0.321385
`Agent11900` en 12000 episodios	0.045189	0.164639	0.649868	0.511786	0.305285

La lectura fue clara:

- 8000 episodios mejoraron claramente al baseline post-fix inicial;
- 12000 episodios no superaron el compromiso alcanzado por `Agent7250`;
- eso indico que el problema ya no era simplemente “mas episodios”.

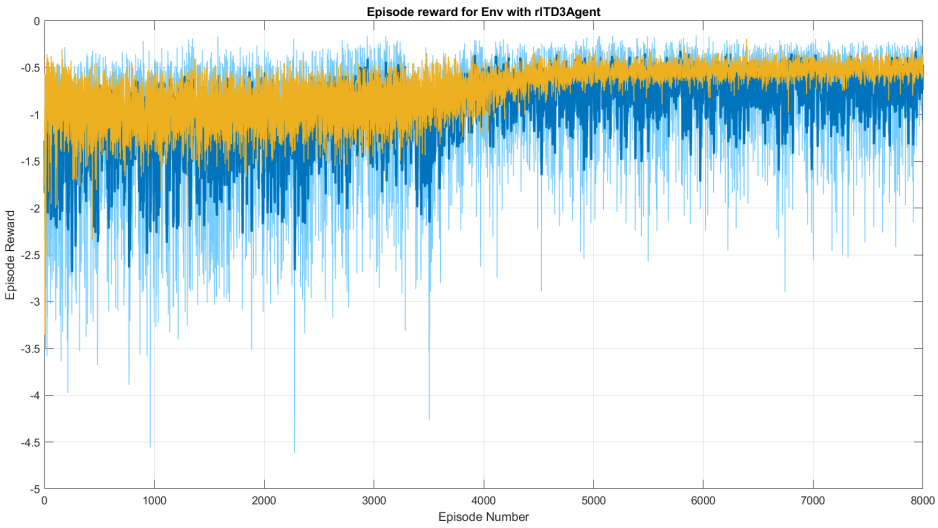

Figura 1: Progreso de entrenamiento de la corrida valida de 8000 episodios. Esta fue la base del benchmark `Agent7250`.

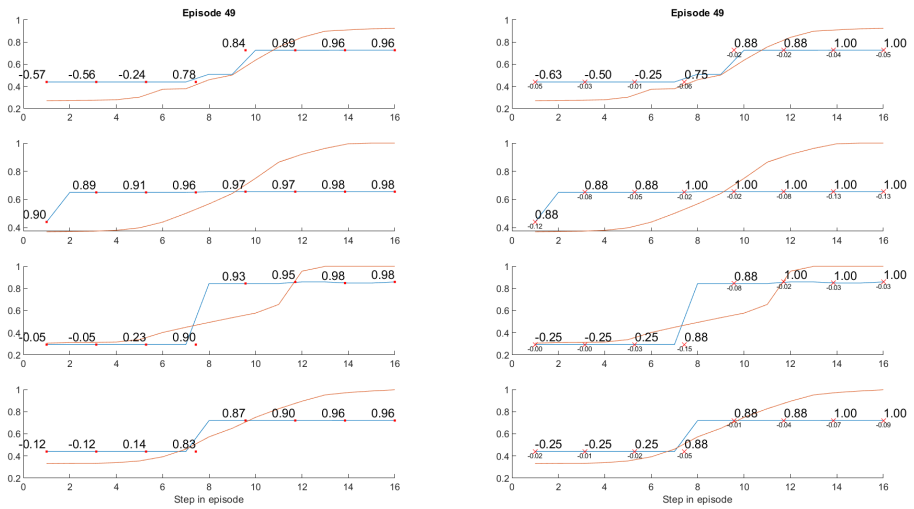

Figura 2: Ejemplo visual del benchmark `Agent7250`. El tracking ya es bueno, pero aun aparece una politica relativamente agresiva.

5. Por que fue necesario abrir una fase nueva despues de `Agent7250`

Cuando `Agent7250` quedo fijado como benchmark, el problema ya no era aprender a seguir la referencia desde cero. El problema dominante paso a ser este:

> mantener o mejorar el tracking sin seguir pagando tanto en saturacion, esfuerzo y agresividad residual.

Ese cambio de objetivo fue importante, porque reorganizo toda la fase experimental posterior. A partir de ahi ya no se buscaba "cualquier mejora", sino una mejora de compromiso.

6. Experimentos posteriores a `Agent7250`

6.1. Continuacion con menor exploracion

La primera idea fue continuar `Agent7250` con menos ruido de exploracion. La razon era simple: si la politica ya estaba en una buena region del espacio de parametros, reducir el ruido podia suavizar el control sin destruir el tracking.

Esa linea produjo un candidato interesante, `Agent1100`, con:

- `trackingMSE = 0.0479`,
- `actionL2 = 0.5306`,
- `saturationFraction = 0.3377`,
- `deltaActionL2 = 0.2752`.

La politica fue mas limpia, pero el tracking cayo demasiado. Sirvio como evidencia de que habia margen para suavizar, pero no como reemplazo del benchmark.

6.2. Exploracion threshold-aware

Despues se refino esa misma idea alrededor del umbral del actuador. El razonamiento fue repo-especifico: habia una discontinuidad real entre el umbral de activacion y el primer nivel efectivo de PWM. Se penso que un barrido fino alrededor de ese punto podia abrir una mejora nueva.

No ocurrio. Esa ruta confirmo el mejor casi-acierto de la continuacion con menor exploracion, pero no produjo un agente nuevo superior.

6.3. Regularizacion por saturacion

Tambien se probo una reward con penalizacion explicita de saturacion. Esa idea estaba bien motivada por el problema observado: si la politica saturaba demasiado, parecia razonable castigar explicitamente la cercania al borde del actuador.

El problema fue que esa reward empujo al sistema hacia politicas demasiado timidas. El tracking empeoro mucho aunque bajaran esfuerzo y saturacion. La linea fue metodologicamente util, pero quedo cerrada como resultado negativo.

6.4. Warm-start offline

Otra ruta fue crear un agente nuevo con una fase offline de *warm-start*. La motivacion era introducir un prior de control mas suave antes del entrenamiento online.

De nuevo, la politica resultante quedo demasiado conservadora. La experiencia fue util para confirmar que el proyecto no necesitaba tanto "inventar otra politica", sino encontrar una forma mas segura de corregir una politica ya fuerte.

6.5. Interfaz de accion alineada al actuador

Tambien se probo una deformacion continua de la accion antes del remapeo final, buscando alinear mejor la salida del actor con los bins fisicamente utiles del actuador. La idea era razonable porque el repo si tiene una discontinuidad real entre accion continua y accion efectiva.

El resultado fue el contrario al deseado: la politica exploto esa alineacion de forma agresiva y colapso a bins altos. Mejoro algo de tracking en algunos checkpoints, pero a costa de mucho mas borde y mucho mas esfuerzo.

6.6. Resumen comparativo

Cuadro 2: Lectura corta de las intervenciones posteriores a `Agent7250`

Intervencion	Por que se intento	Resultado
Baja exploracion	Suavizar una politica ya buena sin cambiar reward ni arquitectura	Mejoro suavidad, pero perdio demasiado tracking.
Threshold-aware	Explotar la discontinuidad real del actuador de forma mas fina	No mejoro el casi-acierto previo.
Reward de saturacion	Atacar de forma directa la tendencia a saturar	Sobre-regularizo y genero politicas demasiado timidas.
Warm-start offline	Introducir un prior suave antes del online RL	Muy conservador; no supero el benchmark.
Interfaz de accion alineada	Reducir el hueco entre accion continua y accion efectiva	Demasiado agresiva; empujo mucho al borde.
Residual TD3	Corregir una politica buena sin reemplazarla por completo	Primera mejora aceptable y defendible.

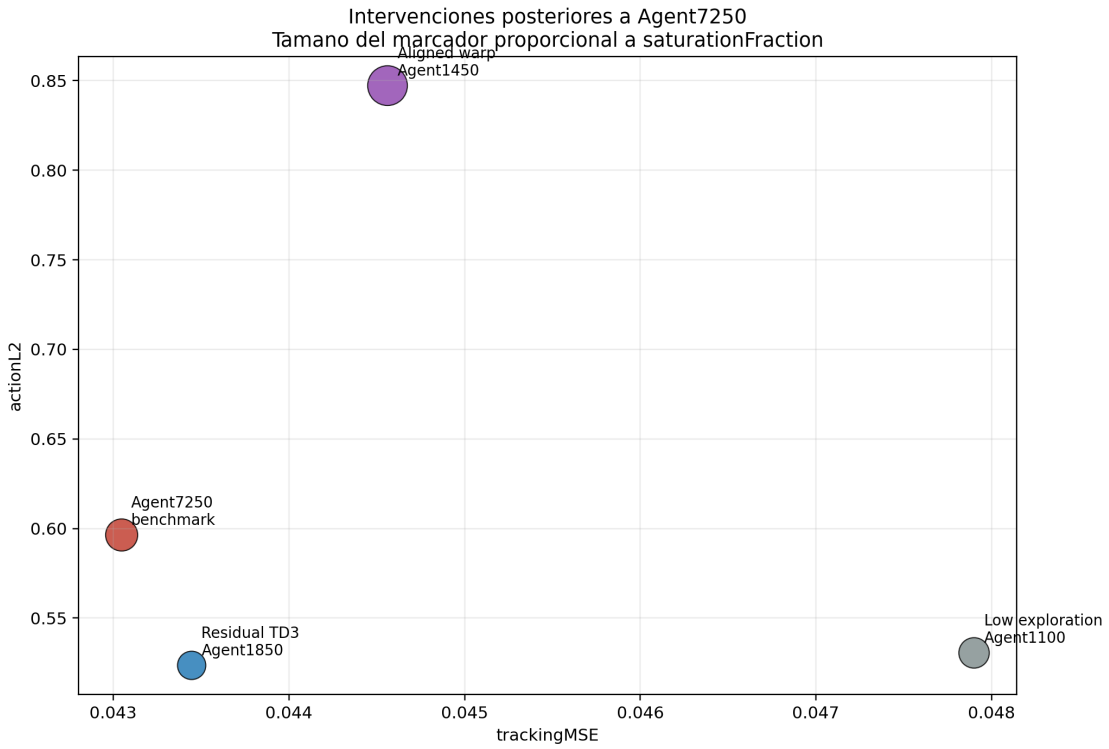

Figura 3: Intervenciones clave posteriores a `Agent7250`. La linea residual es la unica que mejora el compromiso global sin colapsar a un extremo.

7. La fase nueva que si funciono: Residual TD3

7.1. Por que la idea residual tenia sentido

La intuicion de la fase residual fue distinta a la de las lineas anteriores. En vez de seguir moviendo toda la politica, se mantuvo congelada la politica base `Agent7250` y se aprendio solo una correccion pequena por estado. Esa idea esta alineada con *Residual Policy Learning*: cuando ya existe un controlador razonable, a menudo conviene aprender solo el termino faltante [9, 10, 11].

Aplicado a este proyecto, el razonamiento fue:

- `Agent7250` ya resolvia bien el tracking;
- los intentos de mover toda la politica terminaban en un extremo o en otro;
- era mas sensato aprender una correccion pequena alrededor de una base fuerte.

7.2. Que se implemento

La politica residual se definio como:

$$a(s) = \mathrm{clip}\Big(a_{\mathrm{base}}(s) + \alpha_{\mathrm{res}}\, a_{\mathrm{res}}(s, a_{\mathrm{base}}), -1, 1\Big),$$

donde:

- a_{base} es la accion del actor congelado de `Agent7250`;
- a_{res} es la correccion aprendible;
- α_{res} controla cuanto puede apartarse la politica final respecto a la base.

La implementacion se monto dentro del propio actor residual, no en el entorno, para que el critico vea la accion final real. MATLAB y su API de actores deterministas y `dlnetwork` hacian viable esta estrategia [12, 13].

7.3. Piloto residual principal

El piloto principal uso:

- base checkpoint: `Agent7250`,
- $\alpha_{\text{res}} = 0.20$,
- exploracion reducida `0.02 / 0.002 / 1e-4`,
- `2000` episodios.

Su mejor checkpoint auditado fue `Agent1850`, y el test visual de 50 simulaciones dio:

Cuadro 3: Benchmark vs mejor residual del piloto

Metrica	`Agent7250`	Residual `Agent1850`
`trackingMSE`	0.043045	0.043445
`trackingMAE`	0.160336	0.163542
`actionL2`	0.596444	0.523597
`saturationFraction`	0.392086	0.272637
`deltaActionL2`	0.321385	0.299262

Ese resultado fue el primer avance serio de toda la fase posterior al benchmark. El tracking empeoro menos de un 1 %, mientras que esfuerzo y saturacion bajaron con claridad. Bajo la regla del proyecto, el candidato quedo como `ConditionB`.

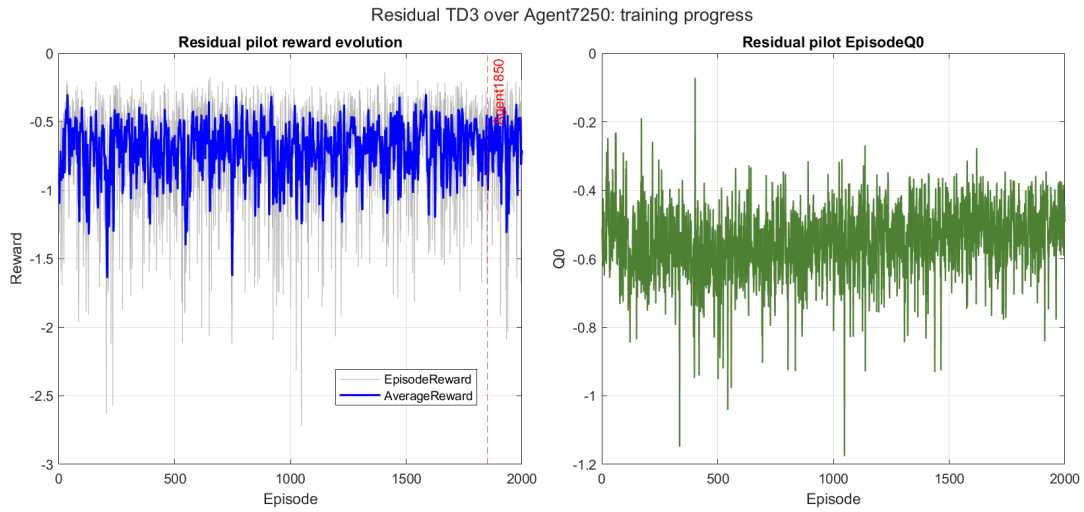

Figura 4: Evolucion de reward y `EpisodeQ0` del piloto residual principal. El mejor checkpoint real no se selecciono por reward final, sino por auditoria y test.

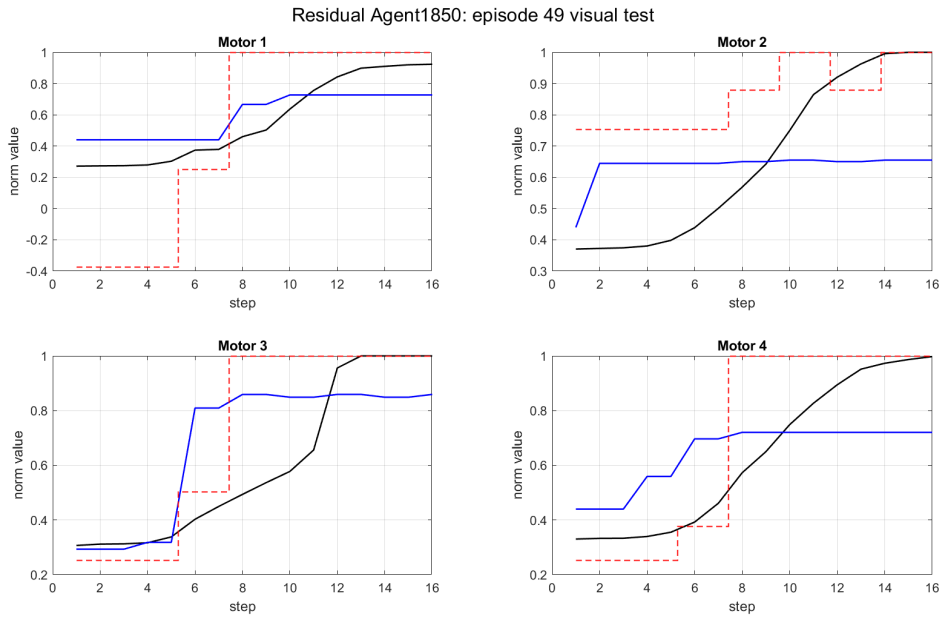

Figura 5: Ejemplo del test visual del residual `Agent1850`. Se observa un seguimiento cercano al benchmark con una accion efectiva mas contenida.

7.4. Por que no se eligio "mas episodios"

Despues del piloto se corrio una extension mas fuerte desde el residual bueno. La hipotesis era razonable: si el mejor checkpoint habia aparecido tarde, tal vez habia margen de mejora con mas entrenamiento.

No ocurrio. La extension no mejoro el compromiso global. La mejor politica de esa continuacion salio peor que el residual original en tracking y esfuerzo. Eso cerro la idea de "seguir entrenando igual" como explotacion principal.

7.5. Que es alpha_res y por que hubo barridos

Una vez demostrado que la idea residual funcionaba, la pregunta correcta ya no era si usar residual o no, sino cual debia ser el tamano de la correccion residual. Ese papel lo cumple α_{res}:

- si α_{res} es muy pequeno, la residual no tiene fuerza suficiente para corregir a la base;
- si es muy grande, la politica final deja de comportarse como una correccion y vuelve a arriesgar agresividad excesiva;
- el mejor valor debe quedar entre esos dos extremos.

Por eso hubo dos barridos:

1. un sweep corto: $\{0.10, 0.20, 0.30\}$;
2. un refinamiento fino: $\{0.18, 0.20, 0.22, 0.25\}$.

7.6. Resultado de los barridos

Cuadro 4: Resultado consolidado de los barridos de alpha_res

alpha_res	trackingMSE	actionL2	saturationFraction	deltaActionL2	Estado final
0.10	0.042506	0.643112	0.498572	0.263525	Rejected
0.18	0.044189	0.665430	0.482118	0.274781	Rejected
0.20	0.043445	0.523597	0.272637	0.299262	ConditionB
0.22	0.043566	0.602889	0.378192	0.276052	Rejected
0.25	0.042503	0.625515	0.451917	0.258078	Rejected
0.30	0.044501	0.533454	0.301742	0.325751	Rejected

La conclusion fue robusta:

- $\alpha = 0.20$ no solo fue el mejor del piloto; sobrevivio tambien al sweep corto y al refinamiento fino;
- $\alpha = 0.10$ fue demasiado debil;
- $\alpha = 0.30$ y la zona alta tendieron a sobrecorregir;
- la zona fina alrededor de 0.20 tampoco produjo un punto mejor.

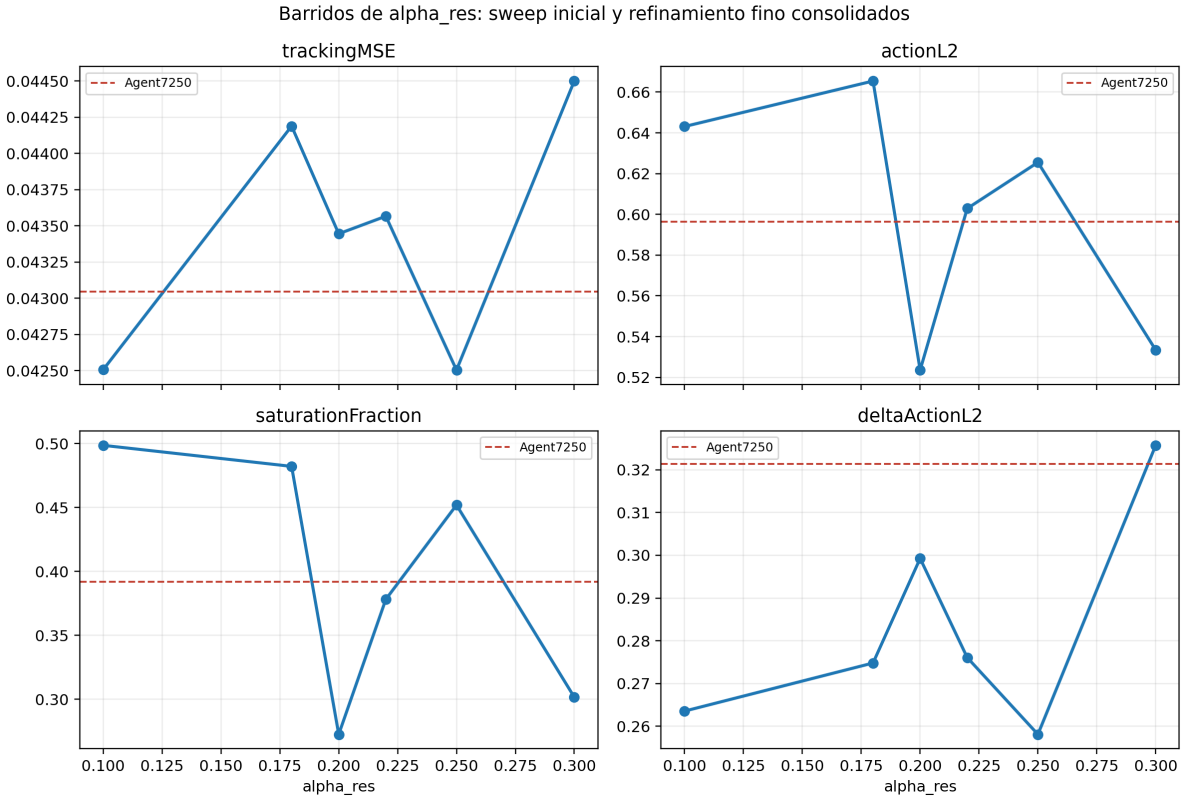

Figura 6: Metricas finales de los barridos de alpha_res. El benchmark `Agent7250` aparece como referencia horizontal.

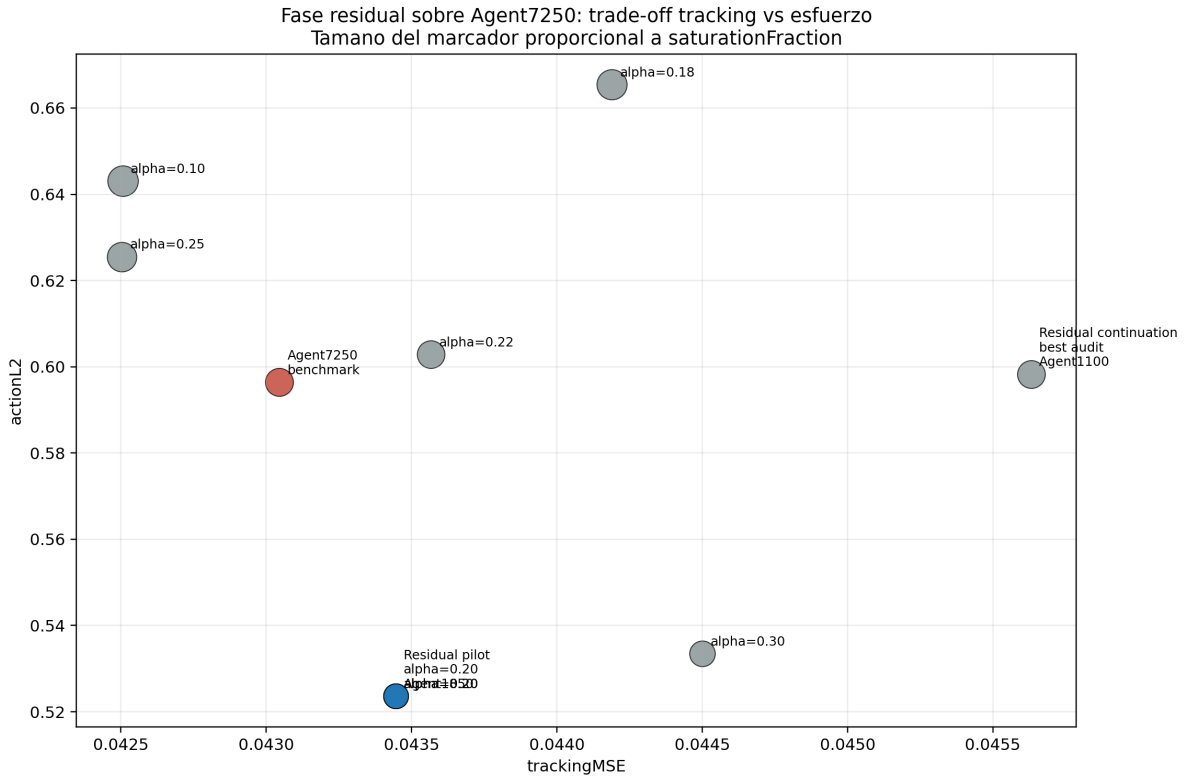

Figura 7: Panorama completo de la fase residual. El mejor compromiso queda en la zona de $\alpha_{\text{res}} = 0.20$.

7.7. Que dicen los diagnosticos residuales

La fase residual no solo produjo una mejora; tambien dio una lectura mas rica del comportamiento del agente. En el mejor caso, la residual no colapso a una politica completamente nueva:

- `ResidualActionL2Mean` fue moderada;
- `BaseToFinalActionDeltaMean` quedo contenida;
- `SignOverrideFractionMean` fue baja.

Eso es importante porque indica que la mejora no vino de romper a la base, sino de corregirla de forma local y controlada. Esa es exactamente la logica que hace atractiva la familia residual [9, 10, 11].

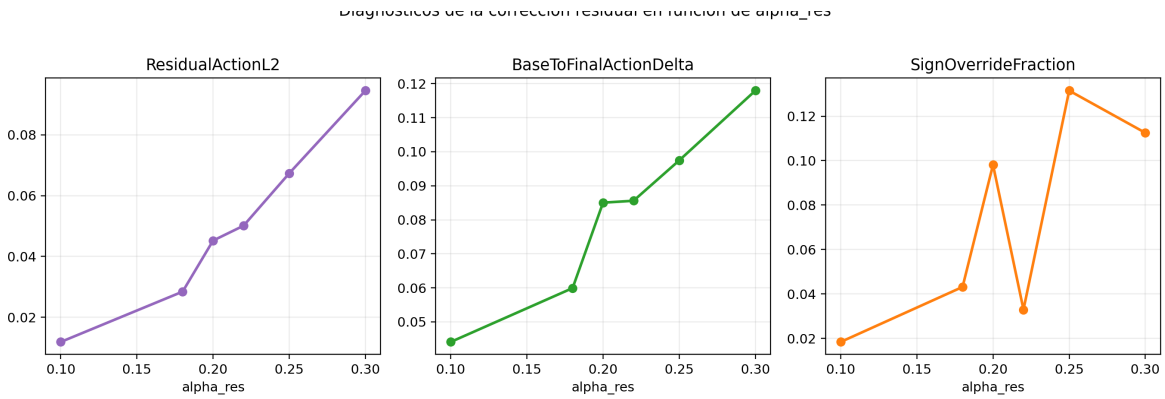

Figura 8: Diagnosticos de cuanto se aparta la politica residual respecto a la base. La zona de $\alpha = 0.20$ mantiene una correccion moderada y util.

8. Agente final recomendado

Con la evidencia completa, el agente final recomendado por esta linea es:

- base historica congelada: `Agent7250`,
- mejor mejora encontrada: residual `Agent1850` con $\alpha_{\text{res}} = 0.20$.

La razon tecnica es directa:

- mantiene practicamente el tracking del benchmark;
- reduce de forma clara saturacion y esfuerzo;
- supera al benchmark bajo `ConditionB`;
- resistio tanto la continuacion como los barridos de α.

9. Conclusion final

La linea del proyecto puede resumirse asi:

1. el articulo base justifico TD3 y el control continuo;
2. el repositorio tuvo que depurar reward, accion efectiva y simulador antes de disponer de un benchmark valido;
3. la corrida post-fix de 8000 episodios produjo a `Agent7250` como benchmark serio;
4. las lineas posteriores a `Agent7250` exploraron varias rutas para bajar agresividad sin perder tracking;
5. la unica ruta que produjo una mejora defendible fue Residual TD3;
6. el punto final operativo de esa linea es el residual `Agent1850` con $\alpha_{\text{res}} = 0.20$.

Por tanto, si el proyecto necesita un cierre tecnico claro, la conclusion ya no es solo historica sino operativa:

> el mejor candidato final del proyecto no es el benchmark base `Agent7250`, sino su mejora residual `Agent1850`, obtenida manteniendo la misma observacion, la misma reward y la misma logica de actuacion, pero aprendiendo una correccion pequena y bien regulada sobre la politica base.

Referencias

[1] Jose Gabriel Fuertes Bustos, Carlos Ayala, Lorena Isabel Barona, Marco E. Benalcazar, and Angel Leonardo Valdivieso. Continuous operation of a myoelectric hand prosthesis using the td3 reinforcement learning algorithm. Technical manuscript provided by the user, 2026. Baseline conceptual article used as historical reference for the project.

[2] Scott Fujimoto, Herke van Hoof, and David Meger. Addressing function approximation error in actor-critic methods. https://arxiv.org/abs/1802.09477, 2018. TD3 original paper.

[3] MathWorks. rltd3agent. https://www.mathworks.com/help/reinforcement-learning/ref/rl.agent.rltd3agent.html, 2026. Accessed: 2026-03-12.

[4] MathWorks. rltd3agentoptions. https://www.mathworks.com/help/reinforcement-learning/ref/rl.option.rltd3agentoptions.html, 2026. Accessed: 2026-03-12.

[5] Ziming Chen, Huasong Min, Dong Wang, Ziwei Xia, Fuchun Sun, and Bin Fang. A review of myoelectric control for prosthetic hand manipulation. *Biomimetics*, 8(3):328, 2023.

[6] Christian Done, Jaden Palmer, Kayson Oakey, Atulan Gupta, Constantine Thiros, Janet Franklin, and Marco P. Schoen. Reinforcement learning-driven prosthetic hand actuation in a virtual environment using unity ml-agents. *Virtual Worlds*, 4(4):53, 2025.

[7] Zelin Deng, Yunlong Dong, and Xing Liu. Reward shaping in reinforcement learning for robotic hand manipulation. *Neurocomputing*, 638:130204, 2025.

[8] I Lee, Hoang-Giang Cao, Cong-Tinh Dao, Yu-Cheng Chen, and I-Chen Wu. Gradient-based regularization for action smoothness in robotic control with reinforcement learning. In *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 603–610, 2024.

[9] Tom Silver, Kelsey Allen, Joshua B. Tenenbaum, and Leslie P. Kaelbling. Residual policy learning. https://arxiv.org/abs/1812.06298, 2018. Accessed: 2026-03-26.

[10] Charles Schaff and Matthew R. Walter. Residual policy learning for shared autonomy. https://arxiv.org/abs/2004.05097, 2020. Accessed: 2026-03-26.

[11] Residual Off-Policy RL Project Page. Introducing resfit: Residual fine-tuning with off-policy rl. https://residual-offpolicy-rl.github.io/, 2025. Accessed: 2026-03-26.

[12] MathWorks. rlcontinuousdeterministicactor. https://www.mathworks.com/help/reinforcement-learning/ref/rl.function.rlcontinuousdeterministicactor.html, 2026. Accessed: 2026-03-26.

[13] MathWorks. functionlayer. https://www.mathworks.com/help/deeplearning/ref/nnet.cnn.layer.functionlayer.html, 2026. Accessed: 2026-03-26.